

Rapport E5

Gestion et deploiement d'applications sur un cluster Kubernetes avec Flux GitOps

BTS SIO -- Option SISR

Abidi Mohamed

Aurlom BTS+

Alternance -- Faculte de Pharmacie de Paris (Universite Paris-Saclay)

Annee 2025-2026

Sommaire

1. Contexte
 2. Objectifs
 3. Cahier des charges
 - 3.1. Contraintes techniques
 - 3.2. Contraintes de securite
 - 3.3. Flux de deploiement GitOps
 4. Solution et Architecture
 - 4.1. Description de l'infrastructure
 - 4.2. Schema d'architecture
 5. Prerequis et Environnement
 - 5.1. Environnement materiel et logiciel
 - 5.2. Applications hebergees
 6. Realisation et Configuration
 - 6.1. Structure d'un deploiement Kubernetes standard
 - 6.2. Deploiement de l'application Assidueite
 - 6.3. Deploiement de l'application Pharmexam
 - 6.4. Deploiement de l'application Annuaire
 - 6.5. Mise en place du pipeline CI/CD
 - 6.6. Resolution des incidents de deploiement
 7. Tests et Validation
 - 7.1. Verification des deploiements
 - 7.2. Tests d'acces aux applications
 - 7.3. Validation du pipeline CI/CD
 8. Conclusion
-

1. Contexte

Dans le cadre de mon alternance au sein du service informatique de la Faculte de Pharmacie de Paris (Universite Paris-Saclay), j'interviens sur l'infrastructure d'hebergement des applications internes de la faculte. Cette infrastructure repose sur un cluster Kubernetes gere selon l'approche GitOps avec Flux CD : l'ensemble des configurations de deploiement sont declarees dans un depot Git centralise (`pharmacie/infra/k8s-iac-flux`), et un controleur Flux synchronise automatiquement l'etat du cluster avec le contenu de ce depot.

La faculté développe et maintient plusieurs applications web internes destinées à la gestion de ses activités : suivi de l'assiduité des étudiants, organisation des examens de pharmacie, annuaire du personnel. Ma mission consiste à déployer et maintenir ces applications sur le cluster Kubernetes, en configurant l'ensemble des ressources nécessaires (Deployments, Services, Ingress, ConfigMaps, Secrets, volumes persistants) et en mettant en place les pipelines CI/CD pour automatiser les futures mises à jour.

2. Objectifs

Les objectifs de cette mission sont les suivants :

- Déployer et maintenir les applications internes de la faculté sur le cluster Kubernetes de production : Assiduite, Pharmexam, Annuaire du Personnel.
- Configurer l'ensemble des ressources Kubernetes pour chaque application en respectant les conventions d'infrastructure établies (sécurité, nommage, segmentation).
- Mettre en place les pipelines CI/CD GitLab pour automatiser la construction des images Docker et le déploiement via Flux.
- Rendre chaque application accessible sur son domaine dédié en HTTPS avec un certificat TLS Let's Encrypt valide.
- Diagnostiquer et résoudre les incidents de déploiement en utilisant les outils Kubernetes (`kubectl logs` , `kubectl describe` , `kubectl get events`).

3. Cahier des charges

3.1. Contraintes techniques

- Le cluster Kubernetes est composé de plusieurs nœuds de travail (k-node-004 à k-node-006).
- Les images Docker sont hébergées sur le registry privé GitLab de la faculté (`gitlab.pharmacie.univ-paris5.fr:8443`), protégé par un certificat auto-signé (CA interne RootCAPharma).
- Le stockage persistant est assuré par un SAN Synology via le driver CSI, avec la StorageClass `iscsi-retain` en mode ReadWriteOnce (RWO).
- Chaque application est déployée dans un namespace Kubernetes dédié portant son nom.
- Le reverse proxy Traefik assure le routage HTTP/HTTPS vers les services internes, avec deux instances : `traefik-public` (IP 194.254.93.5) pour les applications accessibles depuis Internet, et `traefik-private` (IP 172.28.10.126) pour les applications internes.

3.2. Contraintes de sécurité

- Les pods doivent s'exécuter en tant qu'utilisateur non-root (`runAsNonRoot: true` , UID/GID 10001) avec un `securityContext` interdisant l'escalade de privilèges (`allowPrivilegeEscalation: false`).
- Le système de fichiers racine des containers est en lecture seule (`readOnlyRootFilesystem: true`) lorsque l'application le permet, avec des volumes `emptyDir` montés sur `/tmp` pour les écritures temporaires.
- Les secrets (mots de passe de bases de données, clés API, credentials LDAP) sont stockés dans des objets Kubernetes `Secret` et injectés via des variables d'environnement.
- L'accès au registry privé GitLab est assuré par un secret `gitlab-credentials` de type `dockerconfigjson` , répliqué automatiquement dans chaque namespace par le contrôleur Reflector.
- Les certificats TLS sont générés automatiquement par cert-manager avec l'émetteur `letsencrypt-prod` .

3.3. Flux de déploiement GitOps

Le déploiement de toutes les applications suit le même modèle GitOps :

1. Le developpeur pousse son code sur le depot GitLab de l'application (ex : `pharmacie/app/assidueite`).
2. Il cree un tag Git semantique (ex : `git tag v1.0.2 && git push origin v1.0.2`).
3. Le pipeline GitLab CI, base sur le template partage `pharmacie/ci-templates` , construit l'image Docker via BuildKit et la pousse sur le registry.
4. Le job `deploy_flux_iac` met automatiquement a jour le tag de l'image dans le depot d'infrastructure `k8s-iac-flux` , via un commit automatique de l'utilisateur `ci-bot@pharmacie.univ-paris5.fr` .
5. Flux detecte le changement dans le depot (polling toutes les 5 minutes) et applique les nouveaux manifestes sur le cluster.
6. Kubernetes deploie la nouvelle version. L'annotation `reloader.stakater.com/auto: "true"` assure le redemarrage des pods en cas de changement de ConfigMap ou Secret.

4. Solution et Architecture

4.1. Description de l'infrastructure

L'infrastructure d'hebergement repose sur les composants suivants :

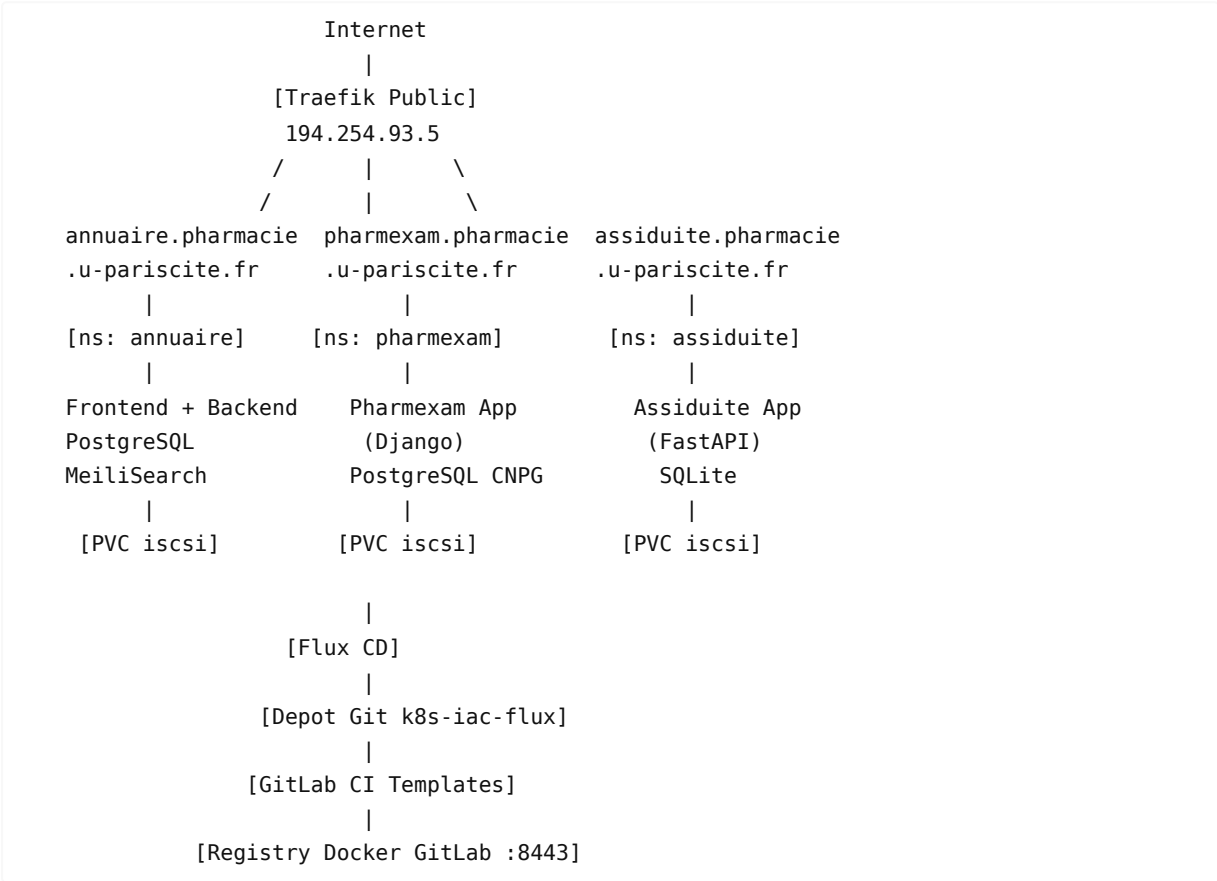
Composant	Role	Detail
Kubernetes	Orchestration des conteneurs	Cluster multi-noeuds
Flux CD	GitOps, synchronisation automatique	Polling 5 min sur <code>k8s-iac-flux</code>
GitLab CI	Build des images Docker	Template partage <code>pharmacie/ci-templates</code>
Traefik	Reverse proxy / Ingress controller	Public (194.254.93.5) et Private (172.28.10.126)
cert-manager	Gestion des certificats TLS	Let's Encrypt (public) et CA interne (prive)
Synology CSI	Stockage persistant	StorageClass <code>iscsi-retain</code> (RWO)
Reflector	Replication de secrets entre namespaces	<code>gitlab-credentials</code> replit depuis <code>kube-system</code>
Reloader	Redemarrage automatique des pods	Sur changement de ConfigMap/Secret

Chaque application suit une structure de manifestes standardisee dans le depot `k8s-iac-flux` :

```
applications/  
  annuaire/  
    namespace.yml  
    deploy.yml  
    svc.yml  
    ingress.yml  
    config.yml  
    secret.yml  
    pvc.yml  
  assidueite/  
    namespace.yml  
    deploy.yml  
    svc.yml  
    ingress.yml
```

```
config.yml
pvc.yml
pharmexam/
  namespace.yml
  deploy.yml
  svc.yml
  ingress.yml
  db.yml
  secret-key.yml
```

4.2. Schema d'architecture



5. Prerequis et Environnement

5.1. Environnement materiel et logiciel

Element	Detail
Cluster Kubernetes	Multi-noeuds (k-node-004 a k-node-006)
Orchestrateur GitOps	Flux CD v2
Registry Docker	GitLab Registry (gitlab.pharmacie.univ-paris5.fr:8443)
CI/CD	GitLab CI + template partage BuildKit
Reverse proxy	Traefik (public + prive)
Certificats TLS	cert-manager + Let's Encrypt

Stockage	Synology CSI, StorageClass iscsi-retain
SMTP	postfix.prod.svc (port 25)
Authentification centralisee	CAS : https://auth.u-paris.fr/idp/profile/cas
Annuaire universitaire	LDAP : ldap://ldap.universite-paris-cite.fr

5.2. Applications hebergees

Application	Technologie	Base de donnees	Domaine
Assidueite	Python / FastAPI	SQLite (PVC)	assidueite.pharmacie.u-pariscite.fr
Pharmexam	Python / Django	PostgreSQL (CNPG)	pharmexam.pharmacie.u-pariscite.fr
Annuaire	Python / FastAPI + React	PostgreSQL + MeiliSearch	annuaire.pharmacie.u-pariscite.fr

6. Realisation et Configuration

6.1. Structure d'un deploiement Kubernetes standard

Chaque application deployee sur le cluster suit un modele commun. Un Deployment Kubernetes type comprend :

- Un **securityContext** non-root (UID/GID 10001) avec interdiction d'escalade de privileges.
- Des **limites de ressources** CPU et memoire pour eviter qu'un pod ne consomme toutes les ressources du noeud.
- Des **probes de sante** (livenessProbe et readinessProbe) sur l'endpoint `/api/health` pour que Kubernetes puisse detecter et redemarrer un pod defaillant.
- Un **imagePullSecrets** referencant le secret `gitlab-credentials` pour acceder au registry prive.
- L'annotation **Reloader** pour le redemarrage automatique sur changement de configuration.

Exemple de structure standard (assidueite) :

```
spec:
  securityContext:
    runAsUser: 10001
    runAsGroup: 10001
    runAsNonRoot: true
  containers:
    - name: run
      image: gitlab.../pharmacie/app/assidueite:v1.0.2
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
      resources:
        limits:
          memory: "512Mi"
          cpu: "500m"
      livenessProbe:
        httpGet:
```

```

    path: /api/health
    port: http
  readinessProbe:
    httpGet:
      path: /api/health
      port: http
  imagePullSecrets:
    - name: gitlab-credentials

```

6.2. Deploiement de l'application Assidueite

L'application Assidueite est un outil de suivi de presence des etudiants, developpe en Python/FastAPI. Son deploiement est le plus simple des trois : une seule image, une base SQLite stockee sur un PVC, et un ConfigMap pour les variables d'environnement.

Fichiers de configuration :

Fichier	Contenu
deploy.yaml	1 Deployment avec strategie RollingUpdate (maxSurge: 0, maxUnavailable: 1)
config.yaml	Variables : BASE_URL, SMTP_HOST, PORT, ADMIN_EMAIL
pvc.yaml	Volume 20 Gi iscsi-retain pour les donnees SQLite
ingress.yaml	Traefik public avec TLS Let's Encrypt
svc.yaml	Service ClusterIP sur le port 8000

Specificite : la strategie `RollingUpdate` avec `maxSurge: 0` est configuree pour eviter le probleme de Multi-Attach sur les volumes iSCSI (RWO). L'ancien pod est supprime avant que le nouveau ne démarre, garantissant qu'un seul pod accede au volume a la fois.

Pipeline CI/CD :

```

include:
  - project: pharmacie/ci-templates
    ref: main
    file: pipelines/container/container.yml

deploy_flux_iac:
  variables:
    AUTO_UPDATE_DIR: "applications/assidueite"

dockerfile_lint:
  allow_failure: true

```

6.3. Deploiement de l'application Pharmexam

Pharmexam est une plateforme d'examens en ligne pour les etudiants en pharmacie, developpee en Python/Django. Son deploiement est plus complexe car il utilise l'operateur CloudNativePG (CNPG) pour la base de donnees PostgreSQL.

Fichiers de configuration :

Fichier	Contenu
deploy.yaml	1 Deployment avec un initContainer pour les migrations Django (python manage.py migrate)
db.yaml	Cluster PostgreSQL CNPG (1 instance, stockage iscsi-retain)
secret-key.yaml	Secret auto-generé par l'opérateur secret-generator
ingress.yaml	Traefik public avec TLS Let's Encrypt
svc.yaml	Service ClusterIP sur le port 8000

Specificite : Le Deployment utilise un **initContainer** qui execute les migrations de base de données Django (`python manage.py migrate --noinput`) avant le démarrage du container principal. Les credentials PostgreSQL sont injectés depuis le secret `pharmexam-pgsql-app` généré automatiquement par l'opérateur CNPG.

Pipeline CI/CD : identique à assidue, avec `AUTO_UPDATE_DIR: "applications/pharmexam"` et une variable supplémentaire `DEV_AUTO_UPDATE_DIR: "applications-dev/pharmexam"` pour l'environnement de développement.

6.4. Deploiement de l'application Annuaire

L'Annuaire du Personnel est l'application la plus complexe à déployer. Elle se compose de quatre services distincts : un frontend React servi par nginx, un backend FastAPI, une base PostgreSQL et un moteur de recherche MeiliSearch.

Fichiers de configuration :

Fichier	Contenu
deploy.yaml	4 Deployments (frontend, backend, postgres, meilisearch)
config.yaml	2 ConfigMaps (backend : DB, LDAP, SMTP, CAS, CORS ; meilisearch : MEILI_ENV)
secret.yaml	Credentials PostgreSQL, cle MeiliSearch, credentials LDAP
pvc.yaml	2 PVC 20 Gi iscsi-retain (PostgreSQL + MeiliSearch)
ingress.yaml	Routage : /api vers backend:8000, / vers frontend:8080
svc.yaml	4 Services ClusterIP

Specificites :

Le Deployment backend utilise **trois initContainers** :

1. `wait-postgres` : attend que PostgreSQL soit disponible (`nc -z annuaire-postgres 5432`)
2. `wait-meilisearch` : attend que MeiliSearch soit opérationnel (`wget http://annuaire-meilisearch:7700/health`)
3. `seed-annuaire` : importe les données initiales du personnel dans la base

Variables d'environnement configurées :

Les valeurs LDAP et CAS ont été déterminées en analysant les configurations des autres applications déployées (appscanetu, traineeship) et le fichier `.env.example` du projet :

Variable	Valeur	Source
LDAP_URL	ldap://ldap.universite-paris-cite.fr	.env.example annuaire
LDAP_BASE_DN	dc=u-paris,dc=fr	.env.example annuaire
CAS_SERVER_URL	https://auth.u-paris.fr/idp/profile/cas	ConfigMap appscanetu/traineeship
SMTP_HOST	postfix.prod.svc	ConfigMap assidueite + deploy ipm
CORS_ORIGINS	https://annuaire.pharmacie.u-pariscite.fr	Adapte pour k8s prod

Correction PGDATA : le volume iSCSI contient un repertoire `lost+found` qui empeche PostgreSQL de s'initialiser. La solution : ajouter `PGDATA: /var/lib/postgresql/data/pgdata` pour que PostgreSQL ecrive dans un sous-repertoire.

Correction MeiliSearch : en production, MeiliSearch exige une cle maitre d'au minimum 16 octets. La cle initiale `masterKey` (9 octets) a ete remplacee par une cle securisee de 44 caracteres.

Pipeline CI/CD : le template partage attend un Dockerfile unique a la racine. Un Dockerfile racine a donc ete cree, reprenant le `backend/Dockerfile` avec les chemins adaptes (`COPY backend/ /app/`). Le frontend reste une image separee construite manuellement.

6.5. Mise en place du pipeline CI/CD

Le pipeline CI/CD de chaque application utilise le meme template partage :

```
include:
  - project: pharmacie/ci-templates
    ref: main
    file: pipelines/container/container.yml

deploy_flux_iac:
  variables:
    AUTO_UPDATE_DIR: "applications/<nom-application>"

dockerfile_lint:
  allow_failure: true
```

Le template gere automatiquement :

- **Stage `pre_build`** : calcul du tag d'image a partir du tag Git ou du commit SHA.
- **Stage `build`** : construction de l'image Docker via `moby/buildkit:rootless` et publication sur le registry GitLab.
- **Stage `test`** : lint du Dockerfile.
- **Stage `deploy`** : mise a jour automatique du tag d'image dans le depot `k8s-iac-flux` par commit du bot CI.

Le workflow de deployment pour toutes les applications est identique :

```
# Modifier le code, commiter, pusher
git push origin main

# Creer un tag pour declencher le build et le deployment
git tag v1.0.3
git push origin v1.0.3
```

```
# Le pipeline build l'image, met a jour k8s-iac-flux, Flux deploie
```

6.6. Resolution des incidents de deployment

Au cours des deploiements, plusieurs incidents ont ete rencontres et resolus :

Incident	Application	Cause	Resolution
ImagePullBackOff (403)	Annuaire	imagePullSecrets absent du Deployment frontend	Ajout de imagePullSecrets: [{name: gitlab-credentials}]
ImagePullBackOff (not found)	Annuaire	Chemin d'image incorrect (annuaire-frontend au lieu de annuaire/frontend)	Correction du chemin dans le registry
ImagePullBackOff (not found)	Annuaire	Tag 0.0.1 inexistant (le tag Git etait v0.0.1)	Ajout du prefixe v
CrashLoopBackOff	Annuaire frontend	nginx.conf contient un proxy_pass vers backend:8000 (service inexistant en k8s)	Suppression du bloc proxy (l'Ingress gere le routage)
CrashLoopBackOff	Annuaire MeiliSearch	Cle maitre trop courte (9 octets, minimum 16)	Remplacement par une cle de 44 caracteres
ContainerCreating bloque	Annuaire postgres/meilisearch	Multi-Attach : PVC iscsi-retain RWO, ancien pod occupe le volume	Suppression des anciens pods ; strategie maxSurge: 0 recommandee
Pipeline CI echoue	Annuaire	Le template CI attend un Dockerfile a la racine du projet	Creation d'un Dockerfile racine
Certificat TLS Docker	Toutes (build local)	Registry GitLab avec CA auto-signee	Extraction du CA et ajout dans /etc/docker/certs.d/

Methodologie de diagnostic : pour chaque incident, les commandes suivantes ont ete utilisees :

```
# Voir l'etat des pods en temps reel
kubectl -n <namespace> get pods -w

# Voir les logs d'un pod
kubectl -n <namespace> logs deploy/<nom> --tail=50

# Voir les evenements detailles d'un pod
kubectl -n <namespace> describe pod <nom-pod>

# Verifier les PVC
kubectl -n <namespace> get pvc

# Verifier les deployments
kubectl -n <namespace> get deploy
```

7. Tests et Validation

7.1. Verification des deploiements

La commande `kubectl get deploy` dans chaque namespace confirme que toutes les applications sont operationnelles :

Assidueite :

NAME	READY	UP-TO-DATE	AVAILABLE
assidueite	1/1	1	1

Pharmexam :

NAME	READY	UP-TO-DATE	AVAILABLE
pharmexam	1/1	1	1

Annuaire :

NAME	READY	UP-TO-DATE	AVAILABLE
annuaire-backend	1/1	1	1
annuaire-frontend	1/1	1	1
annuaire-meilisearch	1/1	1	1
annuaire-postgres	1/1	1	1

7.2. Tests d'accès aux applications

Chaque application est accessible sur son domaine dedie avec un certificat TLS Let's Encrypt valide :

Application	URL	Endpoint de sante	Resultat
Assidueite	https://assidueite.pharmacie.u-pariscite.fr	/api/health	{"status": "healthy"}
Pharmexam	https://pharmexam.pharmacie.u-pariscite.fr	/api/health	{"status": "healthy"}
Annuaire	https://annuaire.pharmacie.u-pariscite.fr	/api/health	{"status": "healthy"}

7.3. Validation du pipeline CI/CD

Le pipeline CI/CD a ete valide pour chaque application en verifiant :

1. **Le build d'image** : la creation d'un tag Git declenche automatiquement la construction de l'image Docker via BuildKit.
2. **La publication** : l'image est pousse sur le registry GitLab avec le tag correspondant (ex : `v1.0.2` , `v1.2.1`).
3. **La mise a jour automatique** : le job `deploy_flux_iac` commit le nouveau tag d'image dans `k8s-iac-flux` (commit automatique par `ci-bot@pharmacie.univ-paris5.fr`).
4. **Le deploiement Flux** : Flux detecte le changement et met a jour le cluster dans les 5 minutes suivantes.

Exemple de commit automatique genere par le pipeline :

```
chore(flux): update gitlab.../pharmacie/app/pharmexam to v1.2.1  
(from pharmacie/app/pharmexam@78a89382)
```

8. Conclusion

La gestion et le déploiement des applications internes de la Faculté de Pharmacie sur le cluster Kubernetes sont pleinement opérationnels. Les objectifs fixés sont atteints :

- **Trois applications sont déployées en production** (Assidue, Pharmexam, Annuaire) avec des niveaux de complexité différents : du déploiement simple à service unique (Assidue) au déploiement multi-services avec base de données et moteur de recherche (Annuaire).
- **L'infrastructure est homogène** : toutes les applications suivent les mêmes conventions de sécurité (non-root, read-only filesystem, limites de ressources), le même modèle de manifests Kubernetes et le même pipeline CI/CD.
- **Le déploiement est automatisé** grâce à l'approche GitOps : un simple `git tag` suivi d'un `git push` suffit pour déployer une nouvelle version, sans intervention manuelle sur le cluster.
- **Les incidents ont été résolus méthodiquement** en s'appuyant sur les outils de diagnostic Kubernetes et en analysant les configurations des applications existantes comme références.

Cette infrastructure GitOps garantit la reproductibilité des déploiements, la traçabilité des changements (chaque modification est un commit Git) et l'automatisation des mises en production. Elle constitue un socle solide pour l'hébergement des futures applications de la faculté.